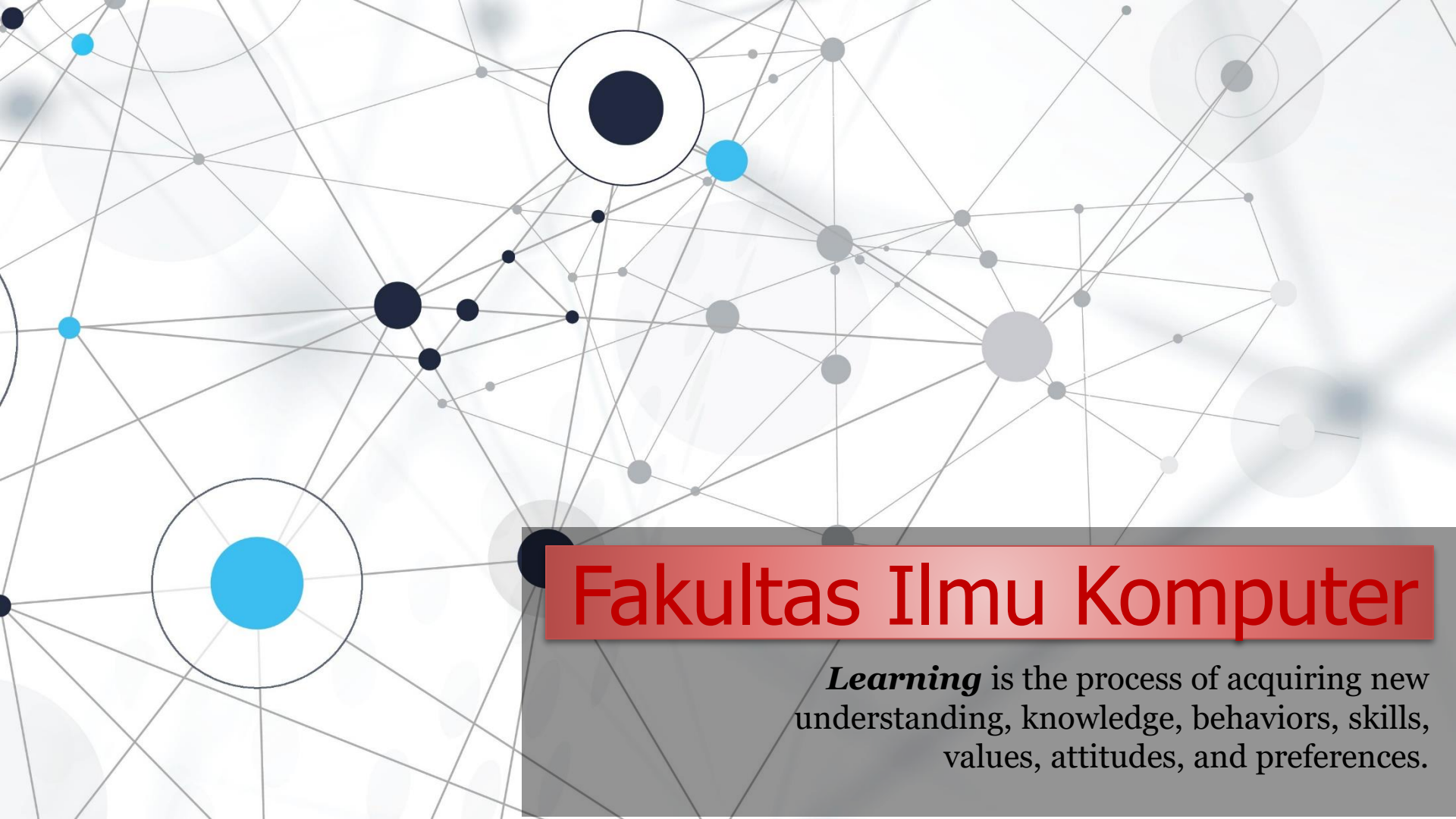


An abstract network diagram with various sized nodes (black, blue, grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

# Fakultas Ilmu Komputer

*Learning* is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

## Collaborative Platform Testing, Managing & Documenting APIs



## Section 3: Containerized Applications with Docker and Kubernetes

---

|                                                               |     |
|---------------------------------------------------------------|-----|
| <b>Chapter 7: Containerizing Your Application with Docker</b> | 215 |
| Technical requirements                                        | 216 |

### *Table of Contents*

---

|                                                            |     |
|------------------------------------------------------------|-----|
| <b>Installing Docker</b>                                   | 216 |
| Registering on Docker Hub                                  | 217 |
| Docker installation                                        | 218 |
| An overview of Docker's elements                           | 223 |
| <b>Creating a Dockerfile</b>                               | 223 |
| Writing a Dockerfile                                       | 224 |
| Dockerfile instructions overview                           | 225 |
| <b>Building and running a container on a local machine</b> | 226 |
| Building a Docker image                                    | 226 |
| Instantiating a new container of an image                  | 228 |
| Testing a container locally                                | 229 |
| <b>Pushing an image to Docker Hub</b>                      | 229 |
| <b>Deploying a container to ACI with a CI/CD pipeline</b>  | 233 |
| The Terraform code for ACI                                 | 234 |
| Creating a CI/CD pipeline for the container                | 235 |
| <b>Summary</b>                                             | 244 |
| <b>Questions</b>                                           | 244 |
| <b>Further reading</b>                                     | 245 |

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green and appear to be from a tree or shrub. The text is centered in the middle of the slide.

# 1 - Collaboration *in* DevOps

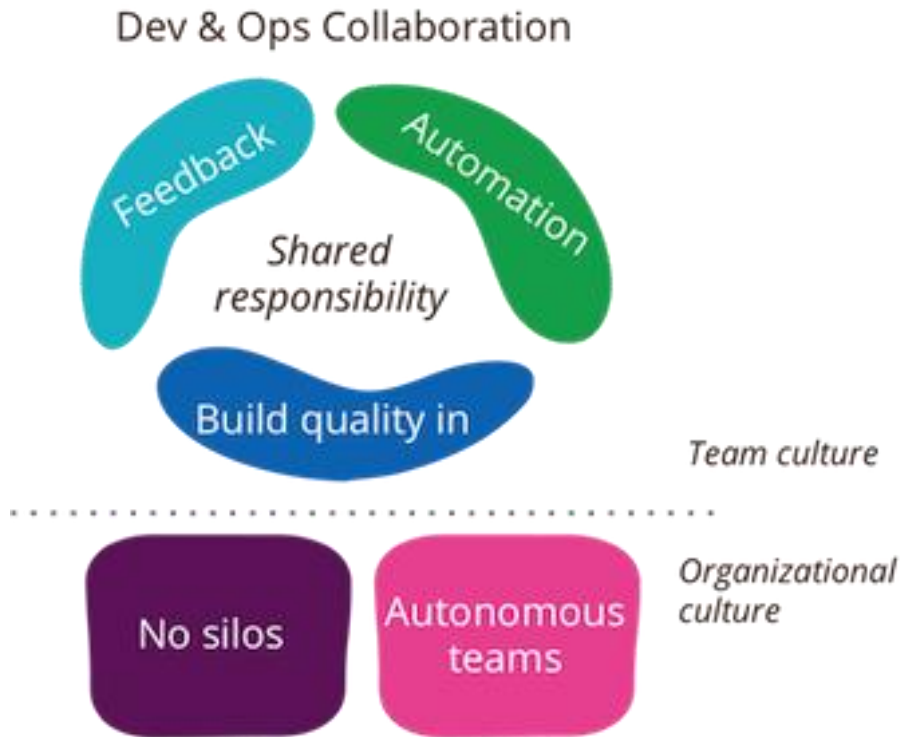
# The 4 Pillars of DevOps



DevOps is a movement focused on improving **communication**, **collaboration** and **integration** between the software development and IT operation teams. DevOps emphasizes on **BRIDGING THE GAP** between these teams by focusing on delivering software faster and creating a success metric.



# What is Collaboration *in* DevOps?



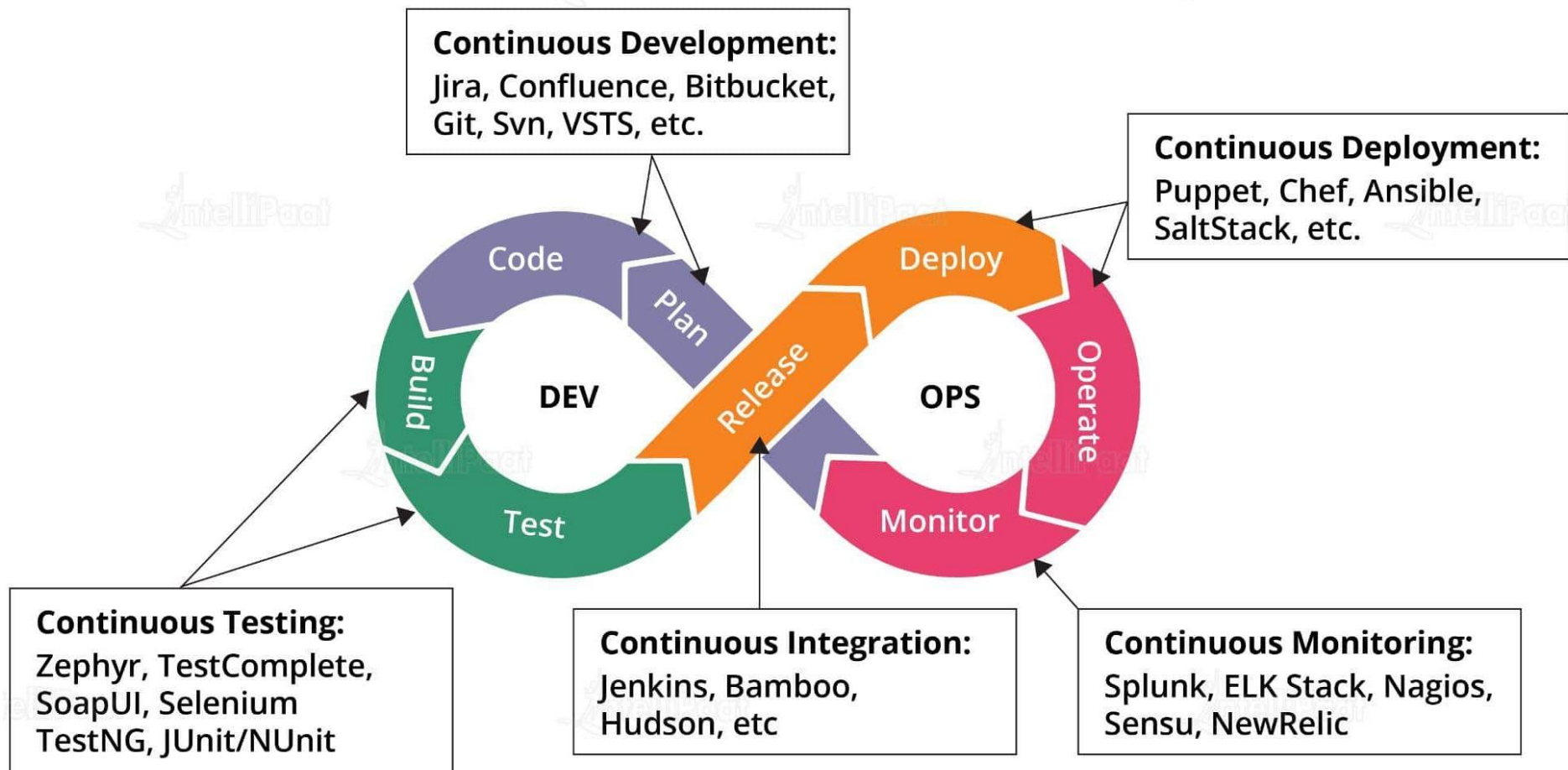
The foundation of DevOps is **A CULTURE OF COLLABORATION** between developers and operations teams, who share responsibilities and combine work.

This makes teams more efficient and saves time related to work handoffs and creating code that is designed for the environment where it runs.

In DevOps, teams work together to accelerate software development and deployment processes, **reduce costs**, and **ensure their organization remains competitive** in the fast-paced digital marketplace.

# What is Collaboration Platform?

**Online collaboration** tools are software that lets **teams work together over the internet**. Examples include **Trello**, **Slack** and **Asana**. The purpose of these tools is **to facilitate communication, collaboration** and **teamwork** for remote teams.



# Effective DevOps Collaboration

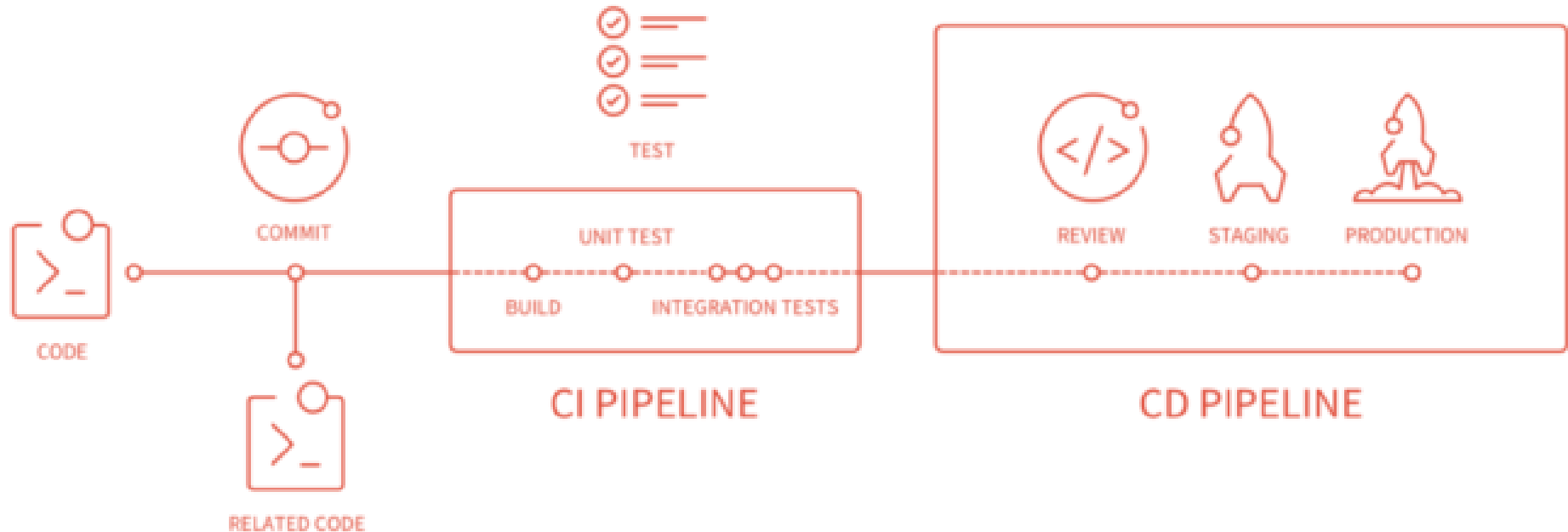
- ❖ There are **DevOps tools** that help us really well during our **Requirements Gathering, Development, Testing**, or even during **Deployment** but apart from these some tools are essential through which team members can **communicate** and also **collaborate**.
- ❖ These DevOps tools are required for the team members to communicate effectively and efficiently, share work, keep up with the tight schedules, and also track the progress of the scheduled tasks.
- ❖ The difficulty in collaboration might grow exponentially when we try to bring both the **DEVELOPMENT TEAM** and **OPERATIONS TEAM** together.

## List of Effective DevOps Collaboration Tools:

1. **Rational DOORS Next Generation (DNG)**
2. **Rational Team Concert (RTC)**
3. **Bitbucket Server**
4. **Pivotal Tracker**
5. **Atlassian Confluence**
6. **Atlassian JIRA**
7. **Micro Focus Project and Portfolio Management**
8. **Microsoft Teams**
9. **Slack**
10. **Basecamp**

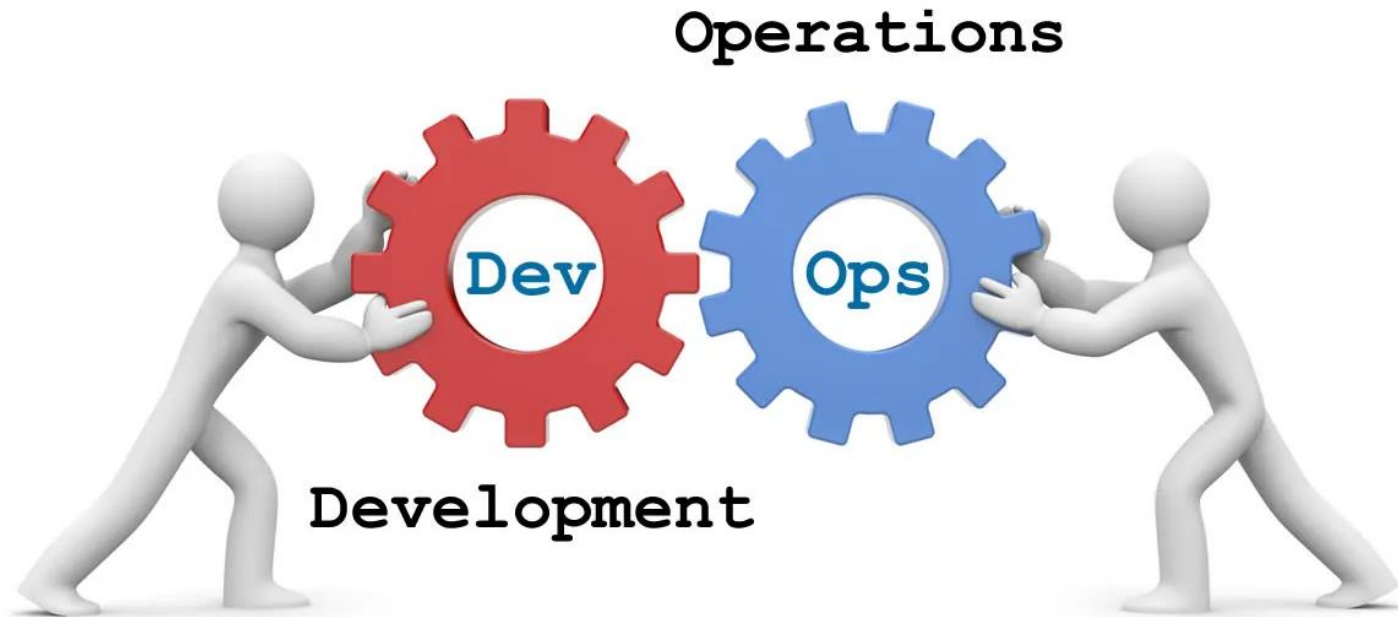


# How do I Collaborate *with* DevOps?



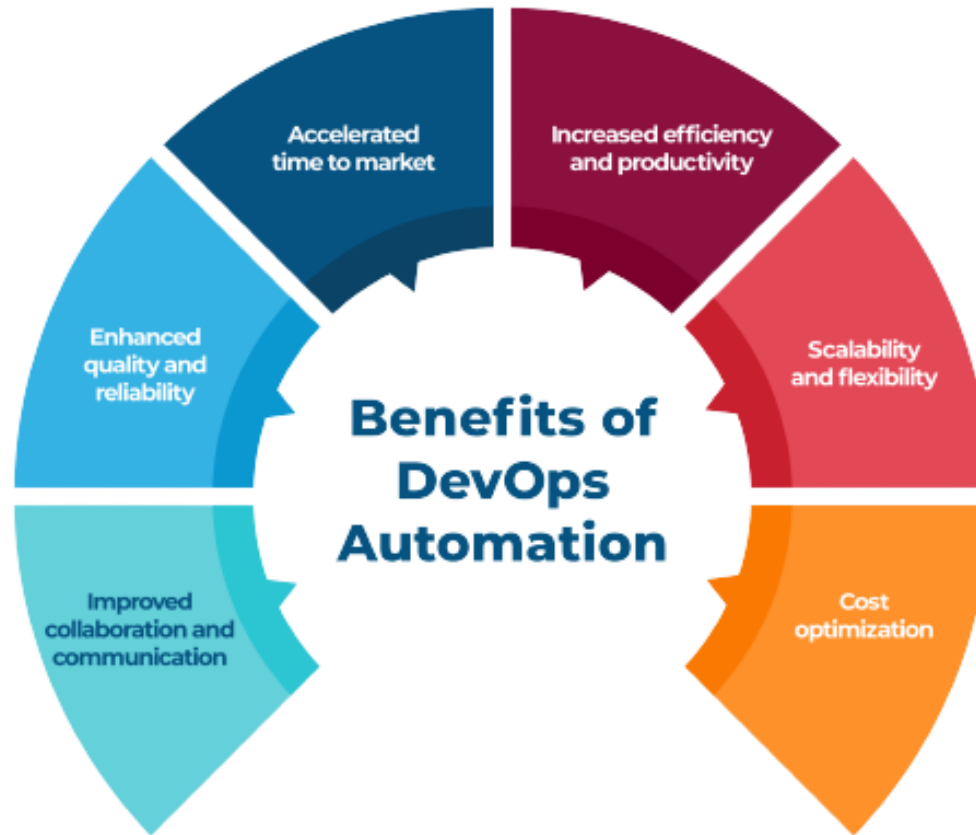
For DevOps teams, we can use **project management tools** to track progress and share files, while **code management tools** can be used to collaborate on code development. It's important to ensure that our team is effectively trained to use all of this technology efficiently.

# Important DevOps Collaboration

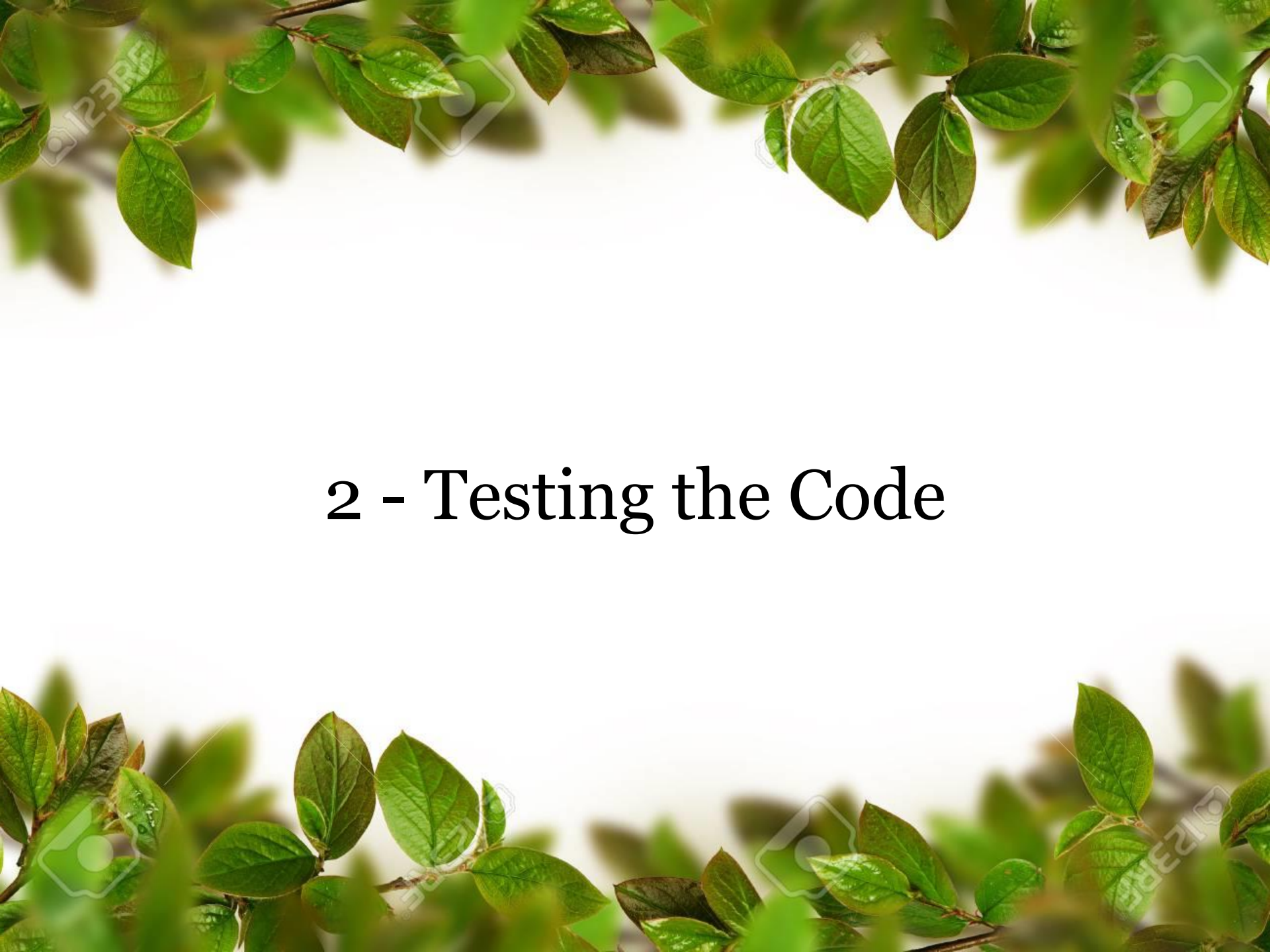


- ❖ Automation is widely considered to be the key to DevOps. The more systems can take care of themselves, the more productive the newly combined development and operational environments can be.
- ❖ **When we hire DevOps engineers, it is important to consider how well they will work with the rest of the team.** Effective collaboration is essential for the success of DevOps initiatives, as it allows teams to share information and resources, and to work together to improve processes and resolve problems.

# Use DevOps Automation



- ❖ **Automation is making all digital work easier**, from CI/CD to security access requests. **Aspects of DevOps collaboration can also be automated**, helping to streamline repetitive and time-consuming tasks.
- ❖ Automating the testing of code, for instance, can free up more time for the sort of creative work in which collaboration is crucial, allowing team members to **focus on more important and strategic work**.

The slide features a white background with green leaves and branches framing the top and bottom edges. The leaves are vibrant green with some visible veins and slight shadows, creating a natural border around the central text.

## 2 - Testing the Code



Community Experience Distilled

# Practical DevOps

Harness the power of DevOps to boost your skill set and make your IT organization perform better

Joakim Verona

**[PACKT]** open source  
PUBLISHING community experience distilled

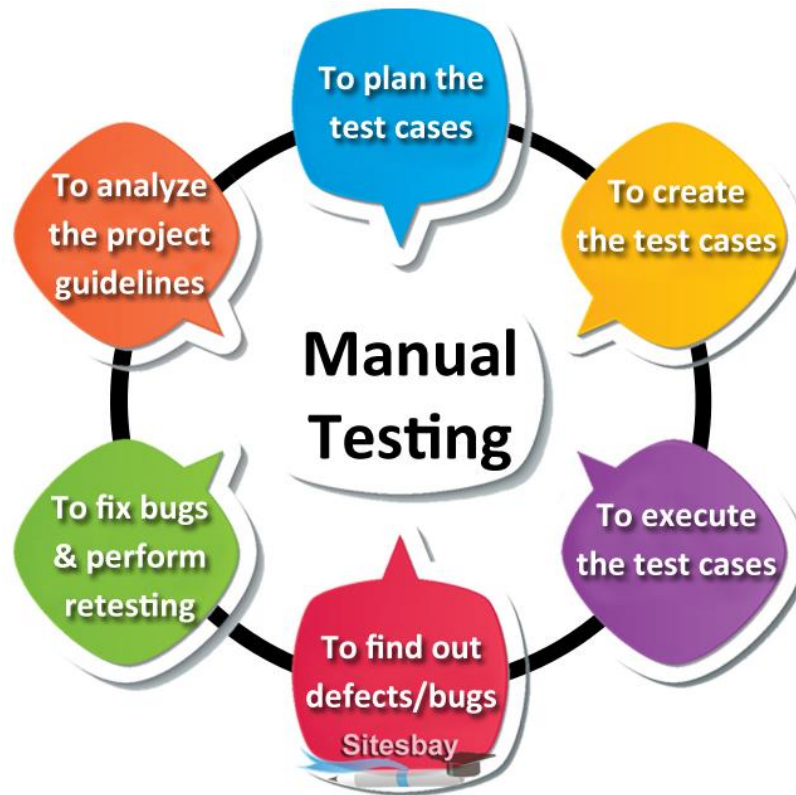
|                                          |           |
|------------------------------------------|-----------|
| <b>Chapter 6: Testing the Code</b>       | <b>83</b> |
| Manual testing                           | 83        |
| Pros and cons with test automation       | 84        |
| Unit testing                             | 86        |
| JUnit in general and JUnit in particular | 87        |
| A JUnit example                          | 89        |
| Mocking                                  | 89        |
| Test Coverage                            | 89        |
| Automated integration testing            | 90        |
| Docker in automated testing              | 91        |
| Arquillian                               | 92        |

[ iii ]

## Table of Contents

|                                       |    |
|---------------------------------------|----|
| Performance testing                   | 92 |
| Automated acceptance testing          | 93 |
| Automated GUI testing                 | 95 |
| Integrating Selenium tests in Jenkins | 96 |





Apa yang kurang dengan manual testing???

# 6

## Testing the Code

If we are going to release our code early and often, we ought to be confident of its quality. Therefore, we need automated regression testing.

In this chapter, some frameworks for software testing are explored, such as JUnit for unit testing and Selenium for web frontend testing. We will also find out how these tests are run in our Continuous Integration server, Jenkins, thus forming the first part of our Continuous Delivery pipeline.

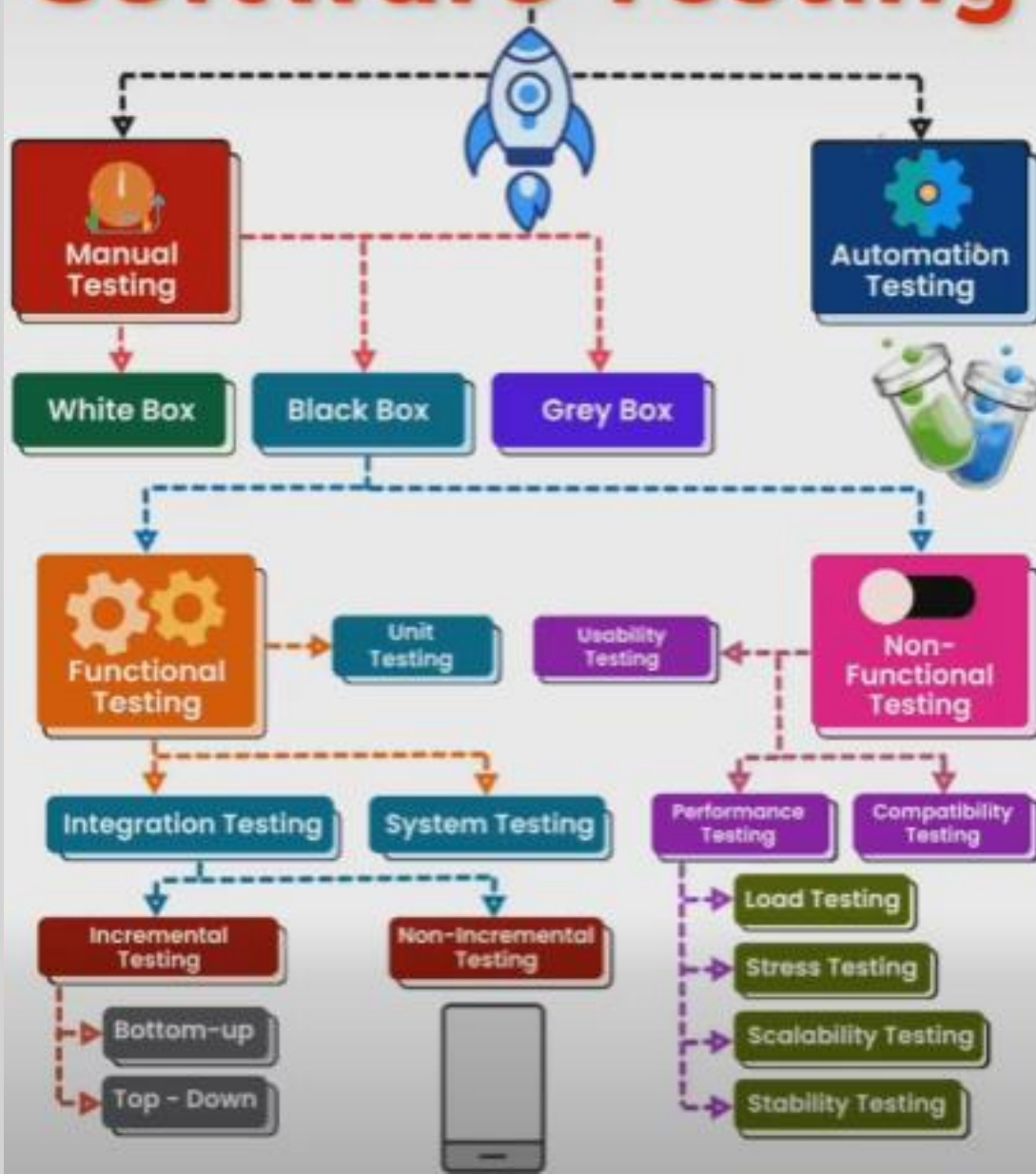
Testing is very important for software quality, and it's a very large subject in itself.

We will concern ourselves with these topics in this chapter:

- How to make manual testing easier and less error prone
- Various types of testing, such as unit testing, and how to perform them in practice
- Automated system integration testing

We already had a look at how to accumulate test data with Sonar and Jenkins in the previous chapter, and we will continue to delve deeper into this subject.

# Types of Software Testing

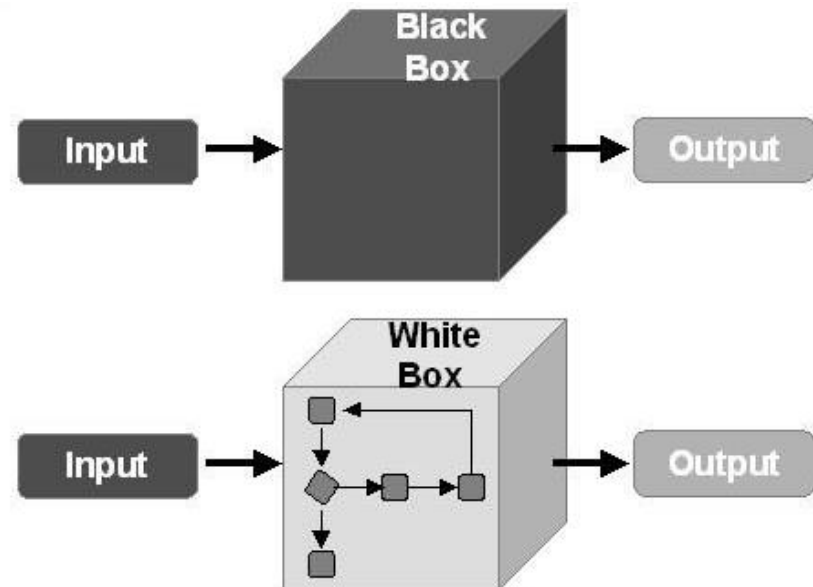
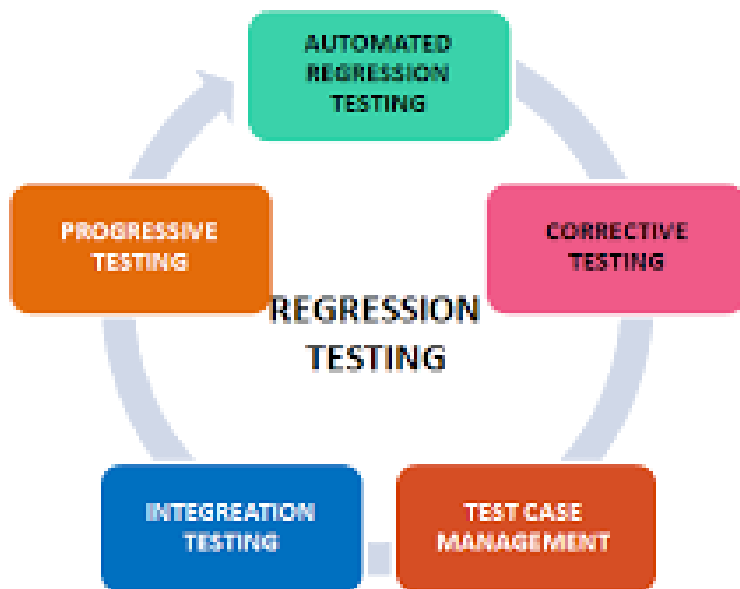


Manual testing merupakan pengujian software dilakukan **secara langsung oleh tester manusia** tanpa bantuan alat otomatis.

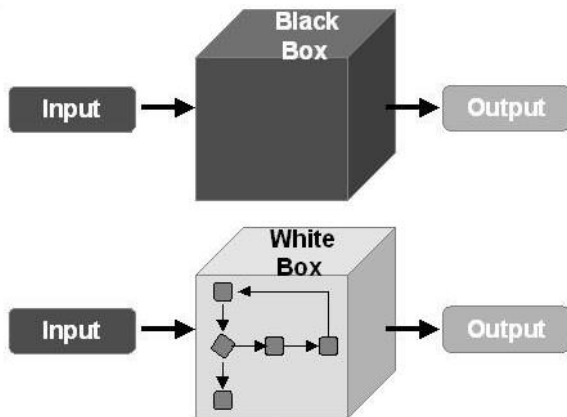
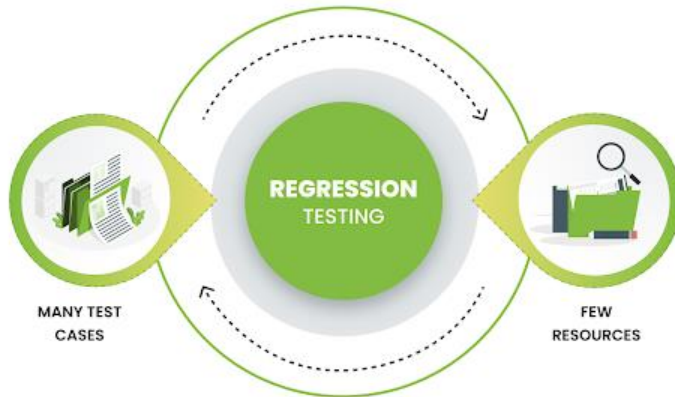
Automation testing merupakan pengujian software dilakukan dengan **menggunakan skrip atau alat otomatis** untuk menjalankan test case.

# Regression Testing

- ❖ **Regression testing** is a **technique that uses a black box to test**. When a code modification in the software is authenticated, it **ensures that the change does not influence the product's existing functionality**.
- ❖ Testers perform regression testing to ensure that the product continues to function properly after adding new functionality, bug fix, or making changes to an existing feature.
- ❖ Software testing is a technique used to ensure that any change or update to a piece of software does not impair the software's general functionality.
- ❖ **Software regression testing is a subset of functional tests that are performed on software.** To ensure that the old functionality of the program is still operational and that the new changes have not introduced any issues, test cases are re-run.



# Automated Regression Testing



- ❖ In simple terms, **automating regression testing** is performing testing over a software product or an application using automated tools and techniques after making an update or a change in the product. The automation process makes the use of test scripts, workflow and other processes within a regression testing technique.
- ❖ Automated regression testing is a software testing process that automatically executes a series of test scripts to ensure that new changes in the code do not break pre-existing functionality (*regression*).
- ❖ The main goal of automated regression testing is to ensure that new changes in the code do not affect or disrupt previously functioning features.

By using automated regression testing, the **software development team can automatically verify that all existing features still work correctly after changes or code additions are made.** This helps ensure software quality is maintained even when development is done quickly and frequently.



# Manual testing

- ❖ Even if test automation has larger potential benefits for DevOps than manual testing, **manual testing will always be an important part of software development**. If nothing else, we will need to perform our tests manually at least once in order to automate them.
- ❖ Acceptance testing in particular is hard to replace, even though there have been attempts to do so. **Software requirement specifications** can be terse and hard to understand even for the people developing the features that implement those requirements. In these cases, **quality assurance people with their eyes on the ball are invaluable and irreplaceable**.
- ❖ **Quality assurance people** are professionals who are responsible for ensuring that the product or service being developed meets established quality standards. They usually work in a department or team called an assurance team or quality team. Their main tasks include developing testing processes, setting quality standards, and ensuring that products or services meet quality requirements before they are released to the market or used by customers.
- ❖ **The things that make manual testing easier are the same things that make automated integration testing easier as well, so there is a synergy to achieve between the different testing strategies.**
- ❖ In order to have happy quality assurance personnel, you need to:
  - Manage test data, primarily the contents of backend databases, so that tests give the same results when you run them repeatedly
  - Be able to make rapid deployments of new code in order to verify bug fixes

“Quality assurance people with  
their eyes on the ball”

Professionals who closely monitor and ensure the quality of a product or service, especially in the context of acceptance testing where they ensure that the product or service fulfils business requirements and user needs well.

# KISS Rule

Meskipun kelihatannya jelas (*manual testing*), namun dalam praktiknya, hal ini bisa jadi sulit. Mungkin kita memiliki basis data produksi besar yang tidak bisa disalin begitu saja ke lingkungan pengujian. Mungkin database tersebut berisi data pengguna akhir yang perlu dilindungi oleh hukum. Dalam kasus ini, kita perlu menghilangkan identifikasi data dan membersihkannya dari detail pribadi sebelum menerapkannya ke lingkungan pengujian.

Each organization is different, so it is not possible to give generally useful advice in this area other than the KISS rule:  
"Keep it simple, stupid."

# Pros & Cons *with* Test Automation

When you talk with people, most are enthusiastic about the prospect of test automation. Imagine all the benefits that await us with it:

- Higher software quality
- Higher confidence that the software releases we make will work as intended
- Less of the monotonous tedium of laborious manual testing.

All very good and desirable things!

- ❖ In practice, though, if you spend time with different organizations with complex multi-tiered products, **you will notice people talking about test automation, but you will also notice a suspicious absence of test automation in practice (*melihat ketiadaan otomatisasi pengujian*)**. Why is that?
- ❖ If you just compile programs and deploy them once they pass compilation, you will likely be in for a bad experience.
- ❖ **Software testing is completely necessary for a program to work reliably (dapat bekerja dengan baik) in the real world. Manual testing is too slow to achieve Continuous Delivery. So, we need test automation to succeed with Continuous Delivery.**

# Why test automation ???

- Cheap tests have lower value.

One problem is that the type of test automation that is fairly cheap to produce, unit testing, typically has lower perceived value than other types of testing. Unit testing is still a good type of testing, but manual testing might be perceived as exposing more bugs in practice. It might then feel unnecessary to write unit tests.

- It is difficult to create test cradles that are relevant to automated integration testing.

While it is not very difficult to write test cradles or test fixtures for unit tests, it tends to get harder as the test cradle becomes more production-like. This can be because of a lack of hardware resources, licensing, manpower, and so on.

- The functionality of programs vary over time and tests must be adjusted accordingly, which takes time and effort.

This makes it seem as though test automation just makes it harder to write software without providing a perceived benefit.

This is especially true in organizations where developers don't have a close relationship with the people working with operations, that is, a non DevOps oriented organization. If someone else will have to deal with your crappy code that doesn't really work as intended, there is no real cost involved for the developers. This isn't a healthy relationship. This is the central problem DevOps aims to solve. The DevOps approach bears this repeating rule: help people with different roles work closer together. In organizations like Netflix, an Agile team is entirely responsible for the success, maintenance, and outages of their service.

- It is difficult to write robust tests that work reliably in many different build scenarios.

A consequence of this is that developers tend to disable tests in their local builds so that they can work undisturbed with the feature they have been assigned. Since people don't work with the tests, changes that affect the test outcomes creep in, and eventually, the tests fail.

Unit testing is the sort of testing that is normally close at heart for developers.

The primary reason is that, by definition, unit testing tests well-defined parts of the system in isolation from other parts.

Thus, they are comparatively easy to write and use.



# Performance Testing

- ❖ Performance testing is an essential part of the development of, for instance, large public web sites.
- ❖ Performance testing presents similar challenges as integration testing. We need a testing system that is similar to a production system in order for the performance test data to be useful to make a forecast about real production system performance.
- ❖ The most commonly used performance test is load testing. With load testing, we measure, among other things, the response time of a server while the performance testing software generates synthetic requests for the server.
- ❖ Apache JMeter is an example of an open source application for measuring performance. While it's simpler than its proprietary counterparts, such as LoadRunner, JMeter is quite useful, and simplicity is not really a bad thing.
- ❖ JMeter can generate simulated load and measure response times for a number of protocols, such as HT, LDAP, SOAP, and JDBC.
- ❖ There is a JMeter Maven plugin, so you can run JMeter as part of your build. JMeter can also be used in a Continuous Integration server.
- ❖ **There is a plugin for Jenkins, called the performance plugin, that can execute JMeter test scenarios. Ideally, the Continuous Integration server will deploy code that has been built to a test environment that is production-like.**

**Jika, testing yang kalian lakukan hanya berapa ada class dan fungsi dari sebuah Java file dengan menggunakan Junit, mungkin itu terkesan mudah saja.**

Apa perbedaan functionality testing dan GUI testing? Lalu, bagaimana cara melakukan GUI testing? Apakah beda dengan functionality testing?

# Automated GUI testing

## Automated GUI testing

Automating GUI testing has many desirable properties, but it is also difficult. One reason is that user interfaces tend to change a lot during the development phase, and buttons and controls move around in the GUI.

Older generations of GUI testing tools often worked by synthesizing mouse events and sending them to the GUI. When a button moved, the simulated mouse click went nowhere, and the test failed. It then became expensive to keep the tests updated with changes in the GUI.

Selenium is a web UI testing toolkit that uses a different, more effective, approach. The controllers are instrumented with identifiers so that Selenium can find the controllers by examining the document object model (DOM) rather than blindly generating mouse clicks.

Selenium works pretty well in practice and has evolved over the years.

Another method is employed by the Sikuli test framework. It uses a computer vision framework, OpenCV, to help identify controllers even if they move or change appearances. This is useful for testing native applications, such as games.

# Integrating Selenium tests *in* Jenkins

## Integrating Selenium tests in Jenkins

Selenium works by invoking a browser, pointing it to a web server running your application, and then remotely controlling the browser by integrating itself in the JavaScript and DOM layers.

When you develop the tests, you can use two basic methods:

- Record user interactions in the browser and later save the resulting test code for reuse
- Write the tests from scratch using Selenium's test API

Many developers prefer to write tests as code using the Selenium API at the outset, which can be combined with a test-driven development approach.

Regardless of how the tests are developed, they need to run in the integration build server.

This means that you need browsers installed somewhere in your test environment. This can be a bit problematic since build servers are usually headless, that is, they are servers that don't run user interfaces.

It's possible to wrap a browser in a simulated desktop environment on the build server.

A more advanced solution is using Selenium Grid. As the name implies, Selenium Grid provides a server that gives a number of browser instances that can be used by the tests. This makes it possible to run a number of tests in parallel as well as to provide a set of different browser configurations.

You can start out with the single browser solution and later migrate to the Selenium Grid solution when you need it.

There is also a convenient Docker container that implements Selenium Grid.



# Exercise *for* Students

# Docker

Docker adalah platform open-source yang memungkinkan pengembang untuk mengemas, mengirim, dan menjalankan aplikasi dalam lingkungan yang terisolasi yang disebut "kontainer". Kontainer Docker berisi semua yang diperlukan untuk menjalankan aplikasi, termasuk kode, runtime, pustaka, dan dependensi lainnya. Docker memanfaatkan teknologi kontainerisasi untuk memastikan aplikasi berjalan dengan konsisten di berbagai lingkungan, mulai dari komputer pengembang hingga pusat data perusahaan dan cloud publik. Docker sangat populer di kalangan pengembang karena portabilitas, konsistensi lingkungan, dan efisiensi dalam penggunaan sumber daya.

## **Cara Install dan Setup Docker di WINdows:**

[https://www.petanikode.com/docker-install/#google\\_vignette](https://www.petanikode.com/docker-install/#google_vignette)



Buatlah demo Automated testing menggunakan Jenkins. Next meeting students akan diminta untuk melakukan Demo **Automated testing** di dalam class !!!

# END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

